

Week 5 — Bottleneck Busting and Team Dynamics

CSE5IDP Industry Development Project · Presenter: Scott Mann · Online cohort notes

Semester 2, 2026 · Milestone 2

Roughly 1h35, of which the formal lecture runs to about the 1h25 mark; the tail is face-to-face students queuing up with project-specific questions.

Housekeeping

- The trade show is done and considered a semester highlight. People's Choice voting forms are open.
- A **supplementary trade show room** is open until next Thursday for the three or four teams affected by late enrolments or an overwritten booth. Links and durations are on the LMS under "supplementary trade show details".
- Every team should ping its project owner in MS Teams this week with a status update — Scott called this out as "super, super important" at the semester midpoint.

Main theme — bottleneck busting

The core message was that unblocking yourself is a **team responsibility**, not something to escalate and then wait on. Scott worked through two live examples.

Leafy AI (IoT data access)

Four teams were all blocked trying to reach a Raspberry Pi behind the La Trobe network. Scott's fix was to invert the architecture:

- The Pi **publishes** sensor data outward to a third-party endpoint. He demoed ThingSpeak, run by MathWorks, using his own astronomy-tower weather station running since 2017.
- Teams then consume it via a read API key — no network penetration required.
- For the reverse direction (pushing control parameters back in), same inversion: the Pi **pulls** a JSON program plan from an external endpoint and executes it locally. Teams get time-sliced control, one team per day or per week.
- One shared script on the device, one working group of one member per team.

LLM evaluation (token budget)

Directly relevant to anything AI-backed:

- Do not start on frontier models. His analogy: *"You're kind of going to the sports car before learning how to drive."*
- Start with open or locally hosted models — he reckons they deliver 80–85 per cent of frontier capability, which is enough to prove your pipeline works.
- Prove the methodology produces indicative results **first**, then make the case for frontier access. That is a much easier sell than asking at week 5.
- Endless experiment-refine loops on a metered API burn tokens by the million; the university cannot hand out an open credit card.
- He suggested Groq free API keys, Llama models and OpenRouter free tiers as starting points.

- Document your **model selection criteria** — parameter counts, what each excels at, why these four.

Also flagged: do not invent third-party dependencies you do not need. His example — a gamification feature does not need API-level access to anything.

Prototyping recap (from W4)

- **Evolutionary** prototypes feed into the final delivered implementation. **Throwaway** prototypes exist for specification validation and are discarded, with the knowledge rolled forward.
- You are primarily doing evolutionary prototyping in this subject. Quick AI-generated throwaway prototypes are useful for validating requirements with the project owner without a full round trip.
- CASE tools (Figma, UML-to-skeleton generators) still have a place, but AI tooling now outpaces most of them.
- Requirements capture here is 80/20 — get started with enough, fill in as you go. Do **not** slide back into a waterfall shape of requirements, design, implementation, testing across the semester.

Evidence in scrum master presentations. Several teams claimed work without proving it. Acceptable evidence: GitHub activity logs, screenshots of actionable outcomes, recorded or live demos. Leniency applied to the first presentation only.

Team dynamics

Development phases: forming and synchronising (weeks 1–4, uncertainty is normal), then conforming (where you should be now), then storming (he sees this in 60–70 per cent of teams, typically weeks 7–9), then a performance uptick. Roughly 65–70 per cent of the room raised hands for feeling integrated; only one of four online.

Role types — you will map to one or several, and diversity across them is the goal:

Role	Characteristic
Recorder	Communication-centric; watches how the team documents and talks
Gatekeeper	Ensures fairness and equal opportunity to contribute (the classical sense, not the meme)
Devil's advocate	Constantly challenges consensus — Scott's favourite, if not taken too far
Taskmaster / timekeeper	Agenda, clock, accountability; one or two per team is ideal
Encourager	Actively pulls people together; thrives on team interaction

Decision-making styles, worst to best: unilateral, hand clasp (a subgroup of two), majority, unanimous (which can mask pressure to conform), then **consensus**, which is his preferred model. Note his aside: the scrum master facilitates the agile process, they are **not** the decision-maker.

Groupthink was called out as the main risk — it guarantees a pass and caps you at roughly a C. Counters: devil's advocacy, open discussion, bringing in outsiders (including Scott), subgrouping to explore alternatives, and a second weekly innovation or experimentation meeting.

Meetings: one or two per week outside class. Distributed agenda, pre-reading, time-boxed topics, round-robin speaking, minutes (manual or AI note-taking), distributed afterwards, action items with named owners. His warning: unstructured online meetings blow out — a 10pm start finishing at 2am kills team motivation by mid-semester.

He will bring **agile temperature cards** (red, yellow, green, blue) to labs to track team sentiment sprint on sprint.

Assessment — what is next

Tender document — due Sunday 23 August. He recommended bringing a draft, or even just the structure, to the online session with Ali on Wednesday for feedback ahead of the deadline. The tender process itself was covered in the last 20 minutes of the W4 lecture — worth the team watching that segment.

Compendium reflections (from the Q&A tail): one template per sprint, so six sprints means six templates, submitted together at the end of semester. It is individual, and Scott explicitly uses it as his **source of truth** for who actually contributed, so that students carrying 80 per cent of a team get rewarded accordingly.

Final handover, weeks 12–13 — previewed now so you can plan for it. Four components:

1. **System design and implementation report** — cyber threat models, UML, use case, class and architecture diagrams; tech stack. Needs *justified* design decisions covering scalability, flexibility, connectability, usability and encountered limitations. One person copying a design they found online is explicitly not good enough.
2. **System maintenance document** — deployment instructions, hardware and software requirements. An A grade needs comprehensive guidance for ongoing maintenance, since projects live on for years afterward. He suggested a short screen-recorded setup walkthrough as high value.
3. **Implementation quality** — industry-standard patterns and frameworks, security protocols, testing from unit level through user acceptance, automated where possible.
4. **User manual** — end-user perspective, fields, data, screenshots, annotated walkthrough of the user stories.

Practical advice: capture and store artefacts **as you go**. Regenerating them in week 12 costs time you will not have. He also suggested using the **mid-semester break** (between weeks 7 and 8) to have one or two members spend an hour collating everything produced so far — sprints during the break are optional.

Milestone 3 gets covered next week.